Fondamenti dell'Informatica

Appunti di Michele Castenedoli

13 ottobre 2026

Indice

0 Prerequisiti Matematici **1**
- 0.1 Basi di Logica . . . 1
- 0.2 Insiemistica . . . 2

1 Introduzione **7**
- 1.1 Il problema dell'informatica . . . 7
- 1.2 Quante funzioni esistono? . . . 7
- 1.3 Quanti algoritmi esistono? . . . 9
- 1.4 Il teorema di Cantor . . . 9

2 Induzione **11**
- 2.1 Relazioni ben fondate . . . 11
- 2.2 Principio di induzione . . . 11

3 Linguaggi Formali **13**
- 3.1 Alfabeti e stringhe . . . 13
- 3.2 Linguaggi . . . 14
- 3.3 Funzioni come linguaggi . . . 14
- 3.4 Automi a stati finiti deterministici . . . 15

0 Prerequisiti Matematici

0.1 Basi di Logica

0.1.1 Linguaggio del Primo Ordine

Un *linguaggio del primo ordine* è formato da quattro tipi di simboli:

1. Simboli di relazione, indicati con le metavariabili p, q, r;
2. Simboli di funzione, indicati con le metavariabili f, g, h;
3. Simboli di costante, indicati con le metavariabili c, d;
4. Simboli logici:
   - le parentesi (e) e la virgola;
   - un insieme numerabile di *variabili*, indicate con le metavariabili x, y, v;
   - i *connettivi logici* e i *quantificatori* $(\wedge, \vee, \neg, \implies \iff, \forall, \exists)$;
   - il simbolo $\bot$, che indica il falso.

I simboli di relazione e di funzione hanno ciascuno la propria *arità*, ovvero il numero di argomenti.

A partire da questi simboli si costruiscono, in ordine, i termini, le formule atomiche e le formule.

Definizione (Termine). *I* termini *sono definiti per induzione:*

1. *ogni variabile è un termine;*
2. *ogni costante è un termine;*
3. *se f è un simbolo di funzione n-ario e $t_1, \dots, t_n$ sono termini, allora $f(t_1, \dots, t_n)$ è un termine.*

Definizione (Formula atomica). *Le* formule atomiche *sono:*

1. *$p(t_1, \dots, t_n)$, se p è un simbolo di relazione n-ario e $t_1, \dots, t_n$ sono termini;*
2. *il simbolo $\bot$.*

Definizione (Formula). *Le* formule *sono definite per induzione:*

1. *ogni formula atomica è una formula;*
2. *se ϕ è una formula, allora $\neg\phi$ è una formula;*
3. *se ϕ e ψ sono formule, allora $\phi \circ \psi$ è una formula, con $\circ \in \{\wedge, \vee, \implies, \iff\}$;*
4. *se ϕ è una formula e x è una variabile, allora $\forall x\,(\phi)$ e $\exists x\,(\phi)$ sono formule.*

0.1.2 Equivalenze

Si scrive $\phi \equiv \psi$ quando le formule ϕ e ψ sono *equivalenti*, ovvero assumono sempre lo stesso valore di verità.

Due negazioni consecutive si annullano (*doppia negazione*):

$$\neg(\neg\phi) \equiv \phi.$$

Le *leggi di De Morgan* descrivono la negazione di una congiunzione e di una disgiunzione:

$$\neg(\phi \wedge \psi) \equiv (\neg\phi) \vee (\neg\psi), \qquad \neg(\phi \vee \psi) \equiv (\neg\phi) \wedge (\neg\psi).$$

Un'*implicazione* può essere riscritta come disgiunzione, oppure come la sua *contronominale* (l'ultima forma):

$$(\phi \implies \psi) \equiv (\neg\phi) \vee \psi \equiv \big((\neg\psi) \implies (\neg\phi)\big).$$

Infine, la *negazione di un quantificatore* si ottiene scambiandolo con l'altro e negando la formula:

$$\neg\big(\exists x\,(\phi)\big) \equiv \forall x\,(\neg\phi), \qquad \neg\big(\forall x\,(\phi)\big) \equiv \exists x\,(\neg\phi).$$

0.1.3 Abbreviazioni

Per i quantificatori ristretti agli elementi di un insieme S si adottano le seguenti abbreviazioni:

$$(\exists x \in S.\ \phi) \equiv \exists x\,(x \in S \wedge \phi),$$
$$(\forall x \in S.\ \phi) \equiv \forall x\,(x \in S \implies \phi),$$

dove il punto si legge *tale che*.

0.2 Insiemistica

0.2.1 Definizione e Rappresentazione

Un *insieme* è una collezione di elementi senza ripetizioni. Esso può essere rappresentato in due modi. La rappresentazione *intensionale* descrive gli elementi attraverso una loro caratteristica: ad esempio

$$\{x \mid p(x)\}$$

è l'insieme di tutti gli elementi x che rendono vero il predicato p. La rappresentazione *estensionale*, invece, elenca tutti gli elementi dell'insieme, come in $\{1, 2, 3\}$.

Un'altra definizione fondamentale per il resto del corso è quella di sottoinsieme.

Definizione (Sottoinsieme). *$A \subseteq B$, ovvero A è un* sottoinsieme *di B, se ogni elemento di A appartiene anche a B. A è invece un* sottoinsieme proprio *di B, $A \subset B$, se in più B contiene almeno un elemento che non appartiene ad A:*

$$A \subseteq B \iff \forall x\,(x \in A \implies x \in B),$$
$$A \subset B \iff A \subseteq B \wedge \exists x \in B.\ x \notin A.$$

0.2.2 Operazioni tra Insiemi

Le operazioni classiche tra insiemi sono:

- Unione: $A \cup B = \{x \mid x \in A \vee x \in B\}$;
- Intersezione: $A \cap B = \{x \mid x \in A \wedge x \in B\}$;
- Differenza: $A \setminus B = \{x \mid x \in A \wedge x \notin B\}$;
- Complemento: $\overline{A} = \{x \in U \mid x \notin A\}$, dove U è l'*insieme universo*, che contiene tutti gli elementi considerati.

Per ogni $x \in U$ vale quindi $x \in A \iff x \notin \overline{A}$ e, analogamente, $x \in \overline{A} \iff x \notin A$.

0.2.3 Relazioni

Per definire le relazioni è necessario il *prodotto cartesiano*. Dati gli insiemi $A_1, A_2, \ldots, A_n$, il loro prodotto cartesiano è definito come mostrato nella Figura 0.1.

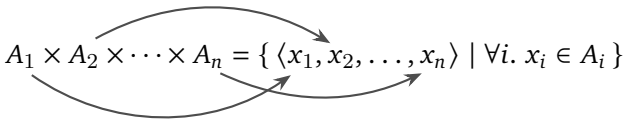


Figura 0.1 *Prodotto cartesiano: la componente x_i di ogni tupla appartiene all'insieme A_i.*

Gli elementi del prodotto cartesiano sono le *tuple* $\langle x_1, x_2, \ldots, x_n \rangle$, in cui la componente i-esima x_i appartiene all'insieme A_i. A differenza degli insiemi, nelle tuple l'ordine è rilevante e gli elementi possono ripetersi: $\langle 1, 2 \rangle \neq \langle 2, 1 \rangle$, mentre $\{1, 2\} = \{2, 1\}$.

Definizione (Relazione binaria). *Dati due insiemi A e B, una* relazione binaria *tra A e B è un sottoinsieme del loro prodotto cartesiano:*

$$R \subseteq A \times B.$$

Se $x_1 \in A$ e $x_2 \in B$ sono in relazione si scrive

$$x_1 \ R \ x_2 \qquad \textit{oppure} \qquad (x_1, x_2) \in R.$$

Le coppie, ovvero le tuple di due elementi, si indicano anche con le parentesi tonde.

0.2.4 Proprietà delle Relazioni

Le proprietà seguenti riguardano le relazioni su un insieme S, ovvero $R \subseteq S \times S$.

Definizione (Relazione riflessiva). *Una relazione R è* riflessiva *se ogni elemento è in relazione con se stesso:*

$$\forall x \in S.\ x \ R \ x.$$

Esempio. La relazione $\subseteq$ tra insiemi è riflessiva, poiché ogni insieme è sottoinsieme di se stesso:

$$\{1, 2, 5, 8\} \subseteq \{1, 2, 5, 8\}.$$

La riflessività deriva dal fatto che il simbolo $\subseteq$ ammette l'uguaglianza; il sottoinsieme proprio $\subset$, invece, non è riflessivo, poiché nessun insieme è sottoinsieme proprio di se stesso. ◇

Definizione (Relazione simmetrica). *Una relazione R è* simmetrica *se la posizione degli elementi nella relazione non è rilevante:*

$$\forall x, y \in S.\ x\ R\ y \implies y\ R\ x.$$

Esempio. L'uguaglianza tra insiemi è simmetrica: se $A = B$, allora anche $B = A$.

$$\underbrace{\{2, 5, 10\}}_{A} = \underbrace{\{2, 5, 10\}}_{B}$$

La relazione $\subseteq$, invece, non è simmetrica: $\{1\} \subseteq \{1, 2\}$, ma $\{1, 2\} \not\subseteq \{1\}$. ◇

Definizione (Relazione transitiva). *Una relazione R è* transitiva *se*

$$\forall x, y, z \in S.\ x\ R\ y \land y\ R\ z \implies x\ R\ z.$$

In altri termini, se x è in relazione con y e y è in relazione con z, allora anche x è in relazione con z, come illustrato nella Figura 0.2.

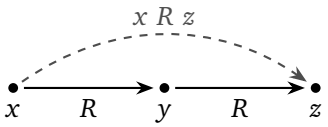


Figura 0.2 *Relazione transitiva: da $x\ R\ y$ e $y\ R\ z$ segue $x\ R\ z$.*

Definizione (Chiusura transitiva). *La* chiusura transitiva *di una relazione $R \subseteq S \times S$ è la relazione R^* definita come il più piccolo insieme $R^* \supseteq R$ tale che R^* sia transitiva: essa si ottiene aggiungendo a R le sole coppie necessarie a renderla transitiva.*

Esempio. Se $R = \{(1, 2), (2, 3)\}$, allora

$$R^* = \{(1, 2), (2, 3), (1, 3)\},$$

dove la coppia $(1, 3)$ è necessaria poiché $1\ R\ 2$ e $2\ R\ 3$. ◇

Definizione (Relazione antisimmetrica). *Una relazione R è* antisimmetrica *se*

$$\forall x, y \in S.\ x\ R\ y \land y\ R\ x \implies x = y.$$

In altri termini, elementi diversi non possono essere scambiati nella relazione: se $x \neq y$ e $x\ R\ y$, allora non può valere $y\ R\ x$.

Esempio. La relazione $\subseteq$ è antisimmetrica, poiché $A \subseteq B$ e $B \subseteq A$ implicano insieme $A = B$. ◇

0.2.5 Tipologie di Relazione

Definizione (Tipi di relazione). *I tipi classici di relazione $R \subseteq S \times S$ sono:*

- *Equivalenza: R è riflessiva, simmetrica e transitiva.*
- *Pre-ordine: R è riflessiva e transitiva. Si tratta di un ordinamento molto debole, poiché manca l'antisimmetria: elementi diversi possono essere in relazione in entrambi i versi.*
- *Ordine parziale: R è un pre-ordine antisimmetrico.* Parziale *significa che non tutti gli elementi sono in relazione tra loro.*
- *Ordine totale: R è un ordine parziale in cui due elementi qualsiasi sono sempre in relazione:*

$$\forall x, y \in S.\ x\ R\ y \vee y\ R\ x.$$

Esempio. La relazione $=$ è di equivalenza, $\subseteq$ è un ordine parziale ($\{1\}$ e $\{2\}$ non sono in relazione in nessuno dei due versi) e $\leq$ sui numeri naturali è un ordine totale. ◇

0.2.6 Elementi Minimali e Massimali

Nel seguito una relazione d'ordine è indicata con $\leq$. La negazione di una relazione si indica barrandone il simbolo: $y \not\leq x$ significa che $y \leq x$ non vale.

Definizione (Elementi minimali e massimali). *Un elemento $x \in S$ è* minimale *rispetto a $\leq$ in S se nessun altro elemento di S è minore o uguale a x:*

$$\forall y \in S \setminus \{x\}.\ y \not\leq x.$$

Analogamente, $x \in S$ è massimale *rispetto a $\leq$ in S se nessun altro elemento di S è maggiore o uguale a x:*

$$\forall y \in S \setminus \{x\}.\ x \not\leq y.$$

In entrambe le definizioni è necessario escludere $y = x$: poiché $\leq$ è riflessiva, vale sempre $x \leq x$, e la condizione $\forall y \in S.\ y \not\leq x$ non sarebbe mai soddisfatta.

Se l'elemento minimale è unico si chiama *minimo*; se l'elemento massimale è unico si chiama *massimo*.[1]

Esempio. In $S = \{\{1\}, \{2\}, \{1, 2\}\}$ con la relazione $\subseteq$, gli elementi minimali sono $\{1\}$ e $\{2\}$, dunque non esiste un minimo, mentre l'unico elemento massimale $\{1, 2\}$ è il massimo. ◇

0.2.7 Funzioni

Definizione (Funzione). *Una funzione è un caso particolare di relazione. Dati due insiemi D, il* dominio*, e C, il* codominio*, una relazione*

$$f \subseteq D \times C$$

è una funzione *da D a C, e si scrive $f\colon D \to C$, se*

$$\forall x \in D.\ \exists!\, y \in C \text{ tale che } f(x) = y.$$

Qui $f(x) = y$ significa $(x, y) \in f$ e il simbolo $\exists!$ si legge *esiste un unico*. Equivalentemente, a ogni elemento del dominio corrisponde uno e un solo elemento del codominio.

[1] Questo vale per gli insiemi finiti. In generale il minimo è l'elemento x tale che $\forall y \in S.\ x \leq y$ e il massimo è l'elemento x tale che $\forall y \in S.\ y \leq x$; in un insieme infinito può esistere un unico elemento massimale che non è il massimo.

Definizione (Immagine). *L'*immagine *di f, $\mathrm{range}(f) \subseteq C$, è l'insieme dei valori del codominio effettivamente assunti da f:*

$$\mathrm{range}(f) = \{\, c \in C \mid \exists d \in D.\ f(d) = c \,\} = \{\, f(d) \in C \mid d \in D \,\}.$$

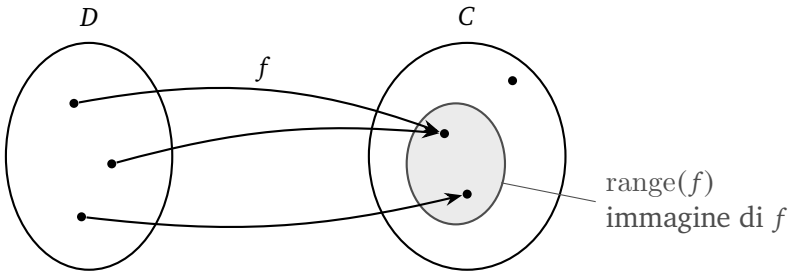


Figura 0.3 *Una funzione $f\colon D \to C$ e la sua immagine* $\mathrm{range}(f)$.

Nella Figura 0.3 ogni elemento di D è origine di esattamente una freccia, come richiesto dalla definizione di funzione, mentre un elemento di C può esserne destinazione più volte oppure mai; in quest'ultimo caso non appartiene all'immagine.

Definizione (Funzione iniettiva). *Una funzione $f\colon D \to C$ è* iniettiva *se elementi diversi del dominio hanno sempre immagini diverse:*

$$\forall x, y \in D.\ f(x) = f(y) \implies x = y.$$

Se f è iniettiva, dunque, due elementi distinti del dominio non possono avere la stessa immagine

Definizione (Funzione suriettiva). *Una funzione $f\colon D \to C$ è* suriettiva *se ogni elemento del codominio è raggiunto da almeno un elemento del dominio:*

$$\forall y \in C.\ \exists x \in D.\ f(x) = y,$$

Definizione (Funzione biettiva). *Una funzione è* biettiva*, o* biezione*, se è sia iniettiva sia suriettiva: ogni elemento del codominio è raggiunto da* esattamente un *elemento del dominio.*

Due insiemi in biezione hanno la stessa cardinalità.

Definizione (Immagine inversa). *Data $f\colon D \to C$ e un sottoinsieme $Y \subseteq C$, l'*immagine inversa *di Y è l'insieme degli elementi del dominio che f manda in Y:*

$$f^{-1}(Y) = \{\, x \in D \mid f(x) \in Y \,\}.$$

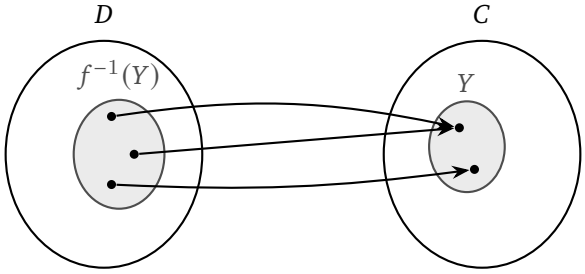


Figura 0.4 *Immagine inversa $f^{-1}(Y)$ di un sottoinsieme $Y \subseteq C$.*

1 Introduzione

1.1 Il problema dell'informatica

Si consideri una generica funzione matematica $f\colon \mathbb{N} \to \mathbb{N}$. Il problema di fondo dell'informatica consiste nello stabilire *quali funzioni siano calcolabili*.

Definizione (Funzione calcolabile).

$$f \textit{ è} \text{ calcolabile} \iff \textit{esiste un algoritmo che la calcola,}$$

dove per algoritmo *si intende una sequenza finita di passi di calcolo.*

Una stessa funzione calcolabile può essere calcolata da più algoritmi diversi, come illustrato nella Figura 1.1.

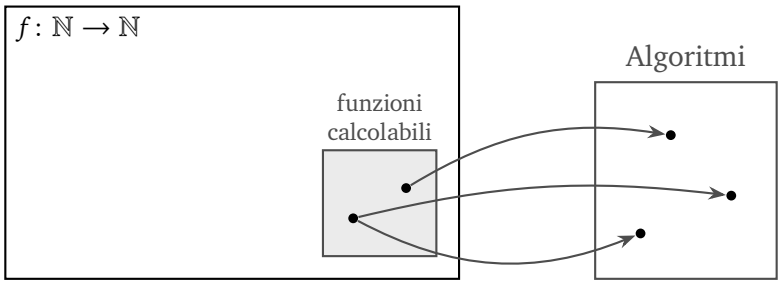


Figura 1.1 *Le funzioni calcolabili sono quelle per cui esiste almeno un algoritmo che le calcola.*

Per stabilire se tutte le funzioni siano calcolabili, si confronta la cardinalità dell'insieme delle funzioni con quella dell'insieme degli algoritmi.

1.2 Quante funzioni esistono?

Una funzione $f\colon \mathbb{N} \to \mathbb{N}$ può essere vista come un *insieme di coppie* in $\mathbb{N} \times \mathbb{N}$:

$$f(2) = 3 \;\mapsto\; (2,3) \qquad f(3) = 5 \;\mapsto\; (3,5) \qquad f(7) = 9 \;\mapsto\; (7,9)$$

Esempio. Una funzione è un insieme infinito di coppie, come

$$\{(0,3),(1,5),(2,2),(3,100),(4,1000),(5,20),(6,2),\dots\},$$

in cui ogni numero naturale compare come primo elemento di *esattamente una* coppia: non è possibile aggiungere un'altra coppia con primo elemento 2, poiché $f(2)$ è già fissato. ◇

Pertanto

$$f \subseteq \mathbb{N}\times\mathbb{N} \iff f \in \mathcal{P}(\mathbb{N}\times\mathbb{N}),$$

dove $\mathcal{P}(X)$ è l'*insieme delle parti* di X, ovvero l'insieme di tutti i suoi sottoinsiemi. Le funzioni da $\mathbb{N}$ in $\mathbb{N}$ sono quindi elementi di $\mathcal{P}(\mathbb{N}\times\mathbb{N})$.

1.2.1 $\mathbb{N}\times\mathbb{N}$ è numerabile

Vale $|\mathbb{N}\times\mathbb{N}| = |\mathbb{N}|$. Le seguenti affermazioni sono equivalenti:

- la cardinalità di $\mathbb{N}\times\mathbb{N}$ è uguale a quella di $\mathbb{N}$;
- l'insieme $\mathbb{N}\times\mathbb{N}$ è *numerabile*;
- esiste una *biezione* (mappatura univoca) tra $\mathbb{N}\times\mathbb{N}$ e $\mathbb{N}$.

Infatti, le coppie possono essere ordinate e numerate percorrendole per diagonali[1] (Figura 1.2):

$$(0,0)\mapsto 0 \quad (0,1)\mapsto 1 \quad (1,0)\mapsto 2 \quad (0,2)\mapsto 3 \quad (1,1)\mapsto 4 \quad (2,0)\mapsto 5 \quad \dots$$

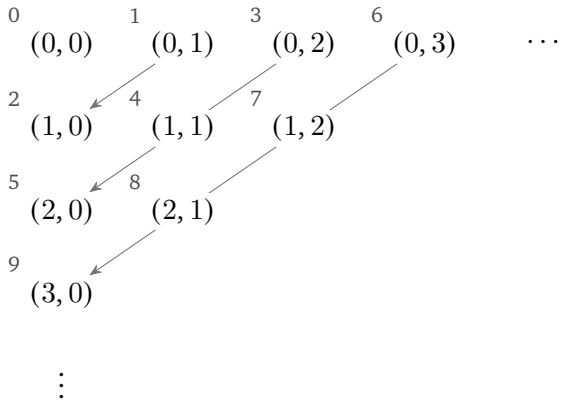


Figura 1.2 *Enumerazione delle coppie di $\mathbb{N}\times\mathbb{N}$ per diagonali.*

Poiché $\mathbb{N}\times\mathbb{N}$ e $\mathbb{N}$ sono in biezione, lo sono anche i loro insiemi delle parti. Quindi

$$|\{f \mid f\colon \mathbb{N}\to\mathbb{N}\}| = |\mathcal{P}(\mathbb{N}\times\mathbb{N})| = |\mathcal{P}(\mathbb{N})|,$$

ovvero esistono *tante funzioni quanti sono i sottoinsiemi di* $\mathbb{N}$.

[1] Non è invece possibile numerare le coppie riga per riga, $(0,0)\mapsto 0$, $(0,1)\mapsto 1$, $(0,2)\mapsto 2$, …: la riga delle coppie che iniziano con 0 è infinita, quindi la coppia $(1,0)$ non riceverebbe mai un numero.

1.3 Quanti algoritmi esistono?

Sia Σ un *alfabeto*, ovvero un insieme finito e non vuoto di simboli: quelli utilizzati per scrivere programmi e algoritmi. Un *programma* è una *stringa* (sequenza di simboli) su Σ, quindi

$$|\text{Programmi}| = |\text{stringhe su } \Sigma| = |\Sigma^*|,$$

Dove * è la *chiusura di Kleene*, ovvero l'insieme di tutte le stringhe ottenute combinando 0 o più volte gli elementi di Σ.
Σ^* è, invece, l'insieme delle stringhe *finite* su Σ, ad esempio:

$$\underbrace{\Sigma = \{a, b\}}_{\text{finito}} \qquad \underbrace{\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, \dots\}}_{\text{infinito}}$$

dove ε è la *stringa vuota*.

Σ^* può essere messo in biezione con $\mathbb{N}$ poiché i suoi elementi sono ordinabili (prima per lunghezza, poi in ordine alfabetico, come nell'esempio): è sufficiente numerarli secondo tale ordine. Quindi Σ^* è numerabile, $|\Sigma^*| = |\mathbb{N}|$, e

$$|\text{Programmi}| = |\Sigma^*| = |\mathbb{N}|.$$

1.4 Il teorema di Cantor

Resta da confrontare $|\mathbb{N}|$, la cardinalità degli algoritmi, con $|\mathcal{P}(\mathbb{N})|$, la cardinalità delle funzioni.

Teorema 1 (di Cantor). *Per ogni insieme S si ha che*

$$|S| < |\mathcal{P}(S)|.$$

Per gli insiemi finiti il teorema è verificabile direttamente, come mostrato nella Tabella 1.1.

Insieme S	Insieme delle parti $\mathcal{P}(S)$	$\lvert S\rvert$	$\lvert\mathcal{P}(S)\rvert$
$\emptyset$	$\{\emptyset\}$	0	1
$\{a, b\}$	$\{\emptyset, \{a\}, \{b\}, \{a, b\}\}$	2	4
$\{a, b, c\}$	$\{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{b, c\}, \{a, c\}, \{a, b, c\}\}$	3	8

Tabella 1.1 *Cardinalità di alcuni insiemi finiti e dei loro insiemi delle parti.*

Dimostrazione. Si suppone per assurdo che $|S| = |\mathcal{P}(S)|$. Allora esiste una funzione biunivoca $f : S \to \mathcal{P}(S)$ che mappa ogni elemento di S in un sottoinsieme di S stesso.

Si consideri l'insieme $A = \{x \in S \mid x \notin f(x)\} \subseteq S$. Dato che $\mathcal{P}(S)$ contiene ogni sottoinsieme di S e A ne è uno, allora $A \in \mathcal{P}(S)$. Pertanto, obbligatoriamente $\exists a \in S \,.\, f(a) = A$. Quindi si hanno due possibilità:

- $a \in A$: allora, per costruzione di A, $a \notin f(a) = A$, $\lightning$.
- $a \notin A$: simmetricamente, per definizione di A, $a \in f(a) = A$, $\lightning$.

Poiché in entrambi i casi si raggiunge una contraddizione, allora $\nexists a \in S \,.\, f(a) = A$. Si conclude, quindi, che A è un elemento privo di controimmagine e, di conseguenza, la cardinalità di S è strettamente inferiore a quella di $\mathcal{P}(S)$. □

In generale

$$|\mathcal{P}(S)| = 2^{|S|}.$$

Per $\mathbb{N}$ vale $|\mathcal{P}(\mathbb{N})| = |\mathbb{R}|$: i sottoinsiemi di $\mathbb{N}$ sono tanti quanti i numeri reali, che *non sono numerabili*. L'*ipotesi del continuo* afferma inoltre che non esiste alcun insieme con cardinalità strettamente compresa tra $|\mathbb{N}|$ e $|\mathbb{R}|$.

Riassumendo:

$$|\text{Programmi}| = |\mathbb{N}| < |\mathcal{P}(\mathbb{N})| = |\{f \mid f \colon \mathbb{N} \to \mathbb{N}\}|.$$

Poiché la cardinalità dei programmi è strettamente inferiore a quella delle funzioni, esistono *funzioni non calcolabili*.

2 Induzione

2.1 Relazioni ben fondate

Definizione (Relazione ben fondata). *Sia A un insieme e $\prec \subseteq A \times A$ una relazione su A, antisimmetrica e transitiva. La relazione $\prec$ è* ben fondata *se non esistono* catene discendenti infinite*, ovvero sequenze infinite $a_0 \succ a_1 \succ a_2 \succ \ldots$ di elementi di A.*

Ne segue che, se A non è vuoto, esistono *elementi minimali*, ovvero elementi di A privi di elementi più piccoli. L'insieme degli elementi minimali di A si indica con $\mathrm{Base}(A)$:

$$\mathrm{Base}(A) = \{\, a \in A \mid \nexists b \in A.\ b \prec a \,\}.$$

La situazione è illustrata nella Figura 2.1.

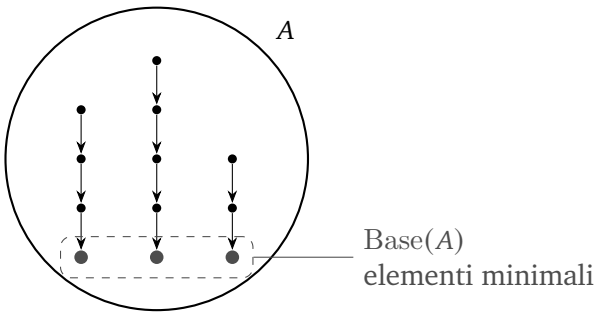


Figura 2.1 *Catene discendenti in un insieme ben fondato e suoi elementi minimali.*

2.2 Principio di induzione

Teorema 2 (Principio di induzione). *Sia $(A, \prec)$ ben fondato e sia Π una proprietà sugli elementi di A. Allora $\forall a \in A.\ \Pi(a)$ se e solo se valgono:*

1. Base dell'induzione*: $\forall a \in \mathrm{Base}(A).\ \Pi(a)$;*
2. Passo induttivo*: per ogni $a \in A \setminus \mathrm{Base}(A)$*

$$\underbrace{\big(\forall b \prec a.\ \Pi(b)\big)}_{\textit{ipotesi induttiva}} \implies \underbrace{\Pi(a)}_{\textit{tesi}}$$

Esempio. Si vuole dimostrare che

$$\forall n \geq 1. \quad \sum_{i=1}^{n} i = \frac{n(n+1)}{2}.$$

Sia $\Pi(n)$ tale uguaglianza. In questo caso $A = \mathbb{N} \setminus \{0\}$ con l'ordine usuale $<$, che è ben fondato, e $\mathrm{Base}(A) = \{1\}$.

Base. Per $n = 1$, $\Pi(1)$ diventa:

$$\sum_{i=1}^{1} i = 1 \qquad \frac{1 \cdot (1+1)}{2} = \frac{1 \cdot 2}{2} = 1.$$

I due membri coincidono, dunque $\Pi(1)$ è verificata.

Passo induttivo. Sia $n \in \mathbb{N} \setminus \{0, 1\}$ e si assuma l'*ipotesi induttiva*, ovvero che $\Pi(m)$ valga per ogni $m < n$. In particolare vale $\Pi(n-1)$, utilizzata nel secondo passaggio:

$$\begin{aligned}
\sum_{i=1}^{n} i &= \sum_{i=1}^{n-1} i + n \\
&= \frac{(n-1)(n-1+1)}{2} + n && \text{(ipotesi induttiva)} \\
&= \frac{(n-1)n + 2n}{2} \\
&= \frac{n(n-1+2)}{2} \\
&= \frac{n(n+1)}{2},
\end{aligned}$$

che è esattamente $\Pi(n)$. ◇

3 Linguaggi Formali

3.1 Alfabeti e stringhe

Definizione (Alfabeto). *Un* alfabeto Σ *è un insieme finito e non vuoto di simboli, ovvero di entità primitive come lettere e caratteri.*

Definizione (Stringa). *Una* stringa*, o* parola*, è una sequenza finita di simboli su un alfabeto* Σ*.*

Come visto nella Sezione 1.3, l'insieme di tutte le stringhe su Σ si indica con Σ^*. Esso contiene sempre la *stringa vuota* ε, priva di simboli, ed è infinito numerabile.

Definizione (Lunghezza di una stringa). *La* lunghezza $|x|$ *di una stringa* x *è il numero di* occorrenze *di simboli al suo interno.*

Si contano le occorrenze e non i simboli: ad esempio $|abba| = 4$, anche se $abba$ contiene solo due simboli distinti. La stringa vuota ha lunghezza $|\varepsilon| = 0$.

Definizione (Prefisso, suffisso e sottostringa). *Sia* $x = a_1 a_2 \cdots a_n$ *una stringa su* Σ*, con* $a_1, \ldots, a_n \in \Sigma$*, e siano* $1 \leq i \leq j \leq n$*. Allora:*

- $a_1 \cdots a_i$*, la parte iniziale di* x*, è un* prefisso *di* x*;*
- $a_j \cdots a_n$*, la parte finale di* x*, è un* suffisso *di* x*;*
- $a_i \cdots a_j$*, una qualsiasi parte contigua di* x*, è una* sottostringa *di* x*.*

Anche la stringa vuota ε è prefisso, suffisso e sottostringa di ogni stringa. Come mostra la Figura 3.1, ogni prefisso e ogni suffisso di x sono anche sue sottostringhe.

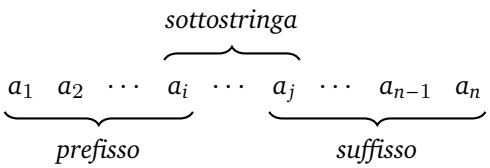


Figura 3.1 *Prefisso* $a_1 \cdots a_i$*, sottostringa* $a_i \cdots a_j$ *e suffisso* $a_j \cdots a_n$ *della stringa* $x = a_1 a_2 \cdots a_n$*.*

3.2 Linguaggi

Definizione (Linguaggio). *Un* linguaggio $\mathcal{L}$ *su un alfabeto* Σ *è un sottoinsieme delle stringhe di* Σ^**, ovvero* $\mathcal{L} \subseteq \Sigma^*$.

Alcuni esempi di linguaggi sono $\emptyset$, dato che è sottoinsieme di qualsiasi insieme, e $\{\varepsilon\}$, il linguaggio che contiene solo la stringa vuota. I due non vanno confusi: $\emptyset$ non contiene alcuna stringa, mentre $\{\varepsilon\}$ ne contiene esattamente una, di lunghezza 0. Entrambi sono linguaggi su qualsiasi alfabeto, mentre Σ^* è il linguaggio di tutte le stringhe su Σ.

Poiché un linguaggio è, di fatto, un insieme, su di esso si possono eseguire le operazioni insiemistiche della Sezione 0.2.2, con Σ^* come insieme universo:

- Complemento di $\mathcal{L} \subseteq \Sigma^*$: $\overline{\mathcal{L}} = \neg\mathcal{L} = \Sigma^* \setminus \mathcal{L}$;
- Unione di $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$: $\mathcal{L}_1 \cup \mathcal{L}_2 = \{x \in \Sigma^* \mid x \in \mathcal{L}_1 \vee x \in \mathcal{L}_2\}$;
- Intersezione di $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*$: $\mathcal{L}_1 \cap \mathcal{L}_2 = \{x \in \Sigma^* \mid x \in \mathcal{L}_1 \wedge x \in \mathcal{L}_2\}$.

Il complemento è quindi sempre relativo a Σ^*. Ad esempio, il complemento del linguaggio vuoto è il linguaggio di tutte le stringhe, mentre quello di $\{\varepsilon\}$ è il linguaggio delle stringhe non vuote:

$$\overline{\emptyset} = \Sigma^* \setminus \emptyset = \Sigma^*, \qquad \overline{\{\varepsilon\}} = \{x \in \Sigma^* \mid x \neq \varepsilon\} = \Sigma^* \setminus \{\varepsilon\}.$$

Altre operazioni invece sono esclusivamente legate ai linguaggi:

- Concatenazione: $\mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^* : \mathcal{L}_1 \cdot \mathcal{L}_2 = \{\ xy\ \in \Sigma^* \mid\ x \in \mathcal{L}_1, y \in \mathcal{L}_2\}$
- Stella di Kleene: $\mathcal{L} \subseteq \Sigma^*$:

$$\mathcal{L}^n = \begin{cases} \mathcal{L}^0 = \{\epsilon\} \\ \mathcal{L}^{n+1} = \mathcal{L} \cdot \mathcal{L}^n \end{cases} \qquad \mathcal{L}^* = \bigcup_{n \in \mathbb{N}} \mathcal{L}^n$$

3.3 Funzioni come linguaggi

Una funzione $f : \mathbb{N} \to \mathbb{N}$ può essere vista come un linguaggio $\mathcal{L}_f$ sull'alfabeto Σ composto da un singolo carattere.
Esempio. Si consideri l'alfabeto $\Sigma = \{1\}$ e una funzione $f : \mathbb{N} \to \mathbb{N}$. Allora:

$$f : \mathbb{N} \to \mathbb{N} \quad \longleftrightarrow \quad \mathcal{L}_f = \{1^{f(x)} \mid x \in \mathbb{N}\},$$

dove 1^n indica la stringa formata da n occorrenze del simbolo 1, con $1^0 = \varepsilon$.

Questo linguaggio permette di porre una domanda con risposta sì o no: data in input una stringa $\sigma \in \Sigma^*$, stabilire se essa appartenga o meno al linguaggio,

$$\sigma \in \Sigma^* \ \longmapsto \begin{cases} \sigma \in \mathcal{L}_f & \text{se } \sigma \text{ appartiene al linguaggio,} \\ \sigma \notin \mathcal{L}_f & \text{se } \sigma \text{ non appartiene al linguaggio.} \end{cases}$$

Si ragiona così su insiemi invece che su funzioni, dove gli elementi dell'insieme dipendono dal calcolo della funzione.

Esempio. Per $f(x) = 2x$ si ottiene

$$\mathcal{L}_f = \{1^{2x} \mid x \in \mathbb{N}\} = \{\varepsilon, 11, 1111, 111111, \dots\},$$

il linguaggio delle stringhe di lunghezza pari su $\{1\}$: ad esempio $1111 \in \mathcal{L}_f$, poiché $f(2) = 4$, mentre $111 \notin \mathcal{L}_f$, poiché nessun x ha $f(x) = 3$. ◇

3.4 Automi a stati finiti deterministici

I linguaggi regolari sono tutti quei linguaggi per cui esiste un automa a stati finiti deterministico che lo riconosce.

Un automa a stati finiti è un modello matematico di un sistema avente input, ed eventualmente output, a valori discreti. Il sistema può essere in uno stato tra un insieme finito di stati possibili. L'essere in uno stato gli permette di tener traccia della storia precedente.

Un automa a stati finiti è rappresentabile da una testina che legge, spostandosi sempre nella stessa direzione e un nastro di lunghezza illimitata contenente dei simboli. La testina può puntare a un certo *stato* q e, a seconda del simbolo w_i, la testina punterà ad un altro stato (oppure sempre allo stesso). La situazione è illustrata nella Figura 3.2.

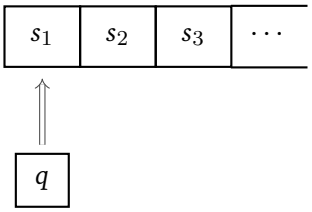


Figura 3.2 *Automa a stati finiti: la testina, nello stato q, legge il primo simbolo s_1 del nastro.*

Quando la lettura dei simboli termina, a seconda dello stato raggiunto dalla testina, l'automa fornisce un risultato di accettazione o di rifiuto della stringa letta.

Il comportamento dell'automa si definisce in maniera univoca mediante una tabella detta *matrice di transizione*.

Esempio. Matrice di Transizione

	a	b
w_0	w_1	w_2
w_1	w_1	w_0
w_2	w_1	w_0

Ma è anche comune la rappresentazione grafica con grafi.

Definizione (Automa Deterministico). *Un automa a stati finiti deterministico (DFA) è una quintupla del tipo $\langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ dove:*

- Q *è un insieme finito di stati;*
- Σ *è un alfabeto;*
- $\delta : Q \times \Sigma \to Q$ *è la* funzione di transizione*;*
- q_0 *è lo stato* iniziale*;*
- $\mathcal{F} \subseteq Q$ *è l'insieme degli stati* finali

Notazione: Useremo p, q, r con o senza pedici per denotare stati, P, Q, R, S per insiemi di stati, a, b con o senza pedici per denotare simboli di Σ, x, y, z, u, v, w sempre con o senza pedici per denotare stringhe.

Poiché la funzione δ acquisisce in ingresso un solo simbolo, è necessario estenderla per poter leggere stringhe. Questo viene eseguito mediante la funzione $\hat{\delta}$, che si ottiene in modo univoco da δ ed è definita come segue:

$$\hat{\delta} \colon Q \times \Sigma^* \to Q, \qquad \begin{cases} \hat{\delta}(q, \varepsilon) = q, \\ \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a). \end{cases}$$

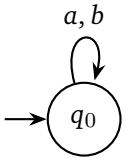
Una stringa x è detta *accettata* o riconosciuta da un DFA $\mathcal{M} = \langle Q, \Sigma, \delta, q_0, \mathcal{F} \rangle$ se $\hat{\delta}(q_0, x) \in \mathcal{F}$. Il linguaggio riconosciuto/accettato da $\mathcal{M}$, denotato come $\mathcal{L}(\mathcal{M})$ è l'insieme delle stringhe accettate, ovvero:

$$\mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* : \hat{\delta}(q_0, x) \in \mathcal{F}\}$$

Definizione (Linguaggio regolare). *Un linguaggio $\mathcal{L}$ è detto* regolare *se è accettato da qualche DFA, ovvero se esiste un DFA $\mathcal{M}$ tale che*

$$\mathcal{L} = \mathcal{L}(\mathcal{M}).$$

Esempio. Il linguaggio vuoto $\mathcal{L} = \emptyset$ è regolare. Lo riconosce il DFA $\mathcal{M}$ della Figura 3.3, con $Q = \{q_0\}$, $\Sigma = \{a, b\}$, stato iniziale q_0 e $\mathcal{F} = \emptyset$.



δ	a	b
q_0	q_0	q_0

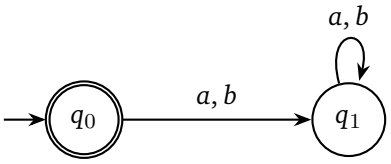
Figura 3.3 *DFA che riconosce il linguaggio vuoto e la sua matrice di transizione.*

Qualunque simbolo legga, l'automa resta in q_0, quindi $\hat{\delta}(q_0, x) = q_0$ per ogni $x \in \Sigma^*$. Poiché $\mathcal{F} = \emptyset$, però, la condizione di accettazione $\hat{\delta}(q_0, x) \in \mathcal{F}$ è sempre falsa: nessuno stato appartiene all'insieme vuoto. Nessuna stringa viene quindi accettata e

$$\mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* \mid \underbrace{\hat{\delta}(q_0, x) \in \mathcal{F}}_{\text{sempre falsa, poichè } \mathcal{F} = \emptyset}\} = \emptyset.$$

Riconoscere $\emptyset$ non significa che l'automa non esista, ma che non accetta alcuna stringa: in generale, ogni DFA con $\mathcal{F} = \emptyset$ riconosce il linguaggio vuoto. ◇

Esempio. Il linguaggio $\mathcal{L} = \{\varepsilon\}$ è regolare. Lo riconosce il DFA $\mathcal{M}$ della Figura 3.4, con $Q = \{q_0, q_1\}$, $\Sigma = \{a, b\}$, stato iniziale q_0 e $\mathcal{F} = \{q_0\}$.



δ	a	b
q_0	q_1	q_1
q_1	q_1	q_1

Figura 3.4 *DFA che riconosce il linguaggio $\{\varepsilon\}$ e la sua matrice di transizione: lo stato finale q_0 è indicato dal doppio cerchio.*

Lo stato iniziale è anche finale, quindi la stringa vuota, che non fa compiere alcuna transizione, viene accettata. Il primo simbolo letto porta invece in q_1, da cui l'automa non esce più:

$$\hat{\delta}(q_0, \varepsilon) = q_0 \in \mathcal{F}, \qquad \hat{\delta}(q_0, x) = q_1 \notin \mathcal{F} \quad \text{per ogni } x \neq \varepsilon.$$

Quindi $\mathcal{L}(\mathcal{M}) = \{\varepsilon\}$. ◇

3.4.1 Come dimostrare che un linguaggio è regolare

Un modo per dimostrare che un linguaggio è regolare è costruire un DFA che lo riconosca. L'uguaglianza $\mathcal{L} = \mathcal{L}(\mathcal{M})$ verrà dimostrata mediante la duplice inclusione $\mathcal{L} \subseteq \mathcal{L}(\mathcal{M})$ e $\mathcal{L}(\mathcal{M}) \subseteq \mathcal{L}$. In particolare:

$$
\begin{aligned}
& \mathcal{L}(\mathcal{M}) \subseteq \mathcal{L} \\
\iff & \forall x \in \Sigma^*.\ \big(x \in \mathcal{L}(\mathcal{M}) \implies x \in \mathcal{L}\big) \\
\iff & \hat{\delta}(q_0, x) \in \mathcal{F} \implies x \in \mathcal{L} && \text{(definizione di } \mathcal{L}(\mathcal{M}), \text{ con } x \text{ arbitrario)} \\
\iff & x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F} && \text{(per contrapposizione)}
\end{aligned}
$$

e

$$
\begin{aligned}
& \mathcal{L} \subseteq \mathcal{L}(\mathcal{M}) \\
\iff & \forall x \in \Sigma^*.\ \big(x \in \mathcal{L} \implies x \in \mathcal{L}(\mathcal{M})\big) \\
\iff & x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F} && \text{(definizione di } \mathcal{L}(\mathcal{M}), \text{ con } x \text{ arbitrario)}
\end{aligned}
$$

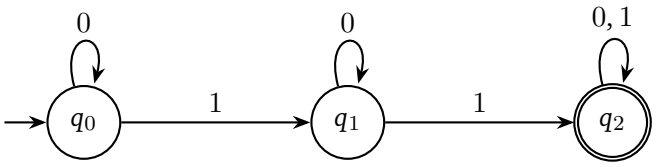
Riassumendo:

$$
\begin{cases}
x \in \mathcal{L} \implies \hat{\delta}(q_0, x) \in \mathcal{F}, \\
x \notin \mathcal{L} \implies \hat{\delta}(q_0, x) \notin \mathcal{F}.
\end{cases}
$$

Dal momento che le condizioni da dimostrare sono analoghe, verranno portate avanti insieme ed è quindi necessario che la base dell'induzione sia la stessa per entrambe. In particolare, è richiesto di trovare una lunghezza tale per cui esistano stringhe in $\mathcal{L}$ e stringhe non in $\mathcal{L}$. I passi sono dunque:

- Base: si considera la lunghezza minima di x e, per tutte le lunghezze inferiori o uguali a quella, si dimostra che valgono entrambe le condizioni;
- Passo induttivo: si suppone che entrambe le condizioni valgano per ogni x tale che $|x| \leq n$ e si dimostra che valgono anche per $|x| = n + 1$.

Esempio. Si consideri il linguaggio $\mathcal{L} = \{x \in \Sigma^* \mid \text{“}x \text{ contiene almeno due } 1\text{”}\}$ definito sull'alfabeto binario $\Sigma = \{0, 1\}$. L'automa a stati finiti che riconosce $\mathcal{L}$ è quello della Figura 3.5, con stato iniziale q_0 e $\mathcal{F} = \{q_2\}$.



δ	0	1
q_0	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_2

Figura 3.5 *DFA che riconosce le stringhe binarie con almeno due* 1 *e la sua matrice di transizione.*

È comunque necessario dimostrare che vengono riconosciute tutte e sole le stringhe di $\mathcal{L}$. In particolare, bisogna dimostrare che

$$\mathcal{L} = \mathcal{L}(\mathcal{M}) = \{x \in \Sigma^* \mid \hat{\delta}(q_0, x) \in \{q_2\}\}.$$

La dimostrazione avviene per induzione. La base non può riguardare *solo* stringhe con meno di due simboli: servono almeno due simboli per arrivare allo stato finale.[1] Quindi:

$$\forall x \in \Sigma^*.\ \big(|x| < 2 \implies x \notin \mathcal{L}\big)$$

Allora:

$$\left.\begin{array}{lll} |x| = 0 & x = \varepsilon \notin \mathcal{L} & \hat{\delta}(q_0, \varepsilon) = q_0 \notin \mathcal{F} \\ |x| = 1 & x = 0 \notin \mathcal{L} & \hat{\delta}(q_0, 0) = q_0 \notin \mathcal{F} \\ & x = 1 \notin \mathcal{L} & \hat{\delta}(q_0, 1) = q_1 \notin \mathcal{F} \\ |x| = 2 & x = 00 \notin \mathcal{L} & \hat{\delta}(q_0, 00) = q_0 \notin \mathcal{F} \\ & x = 01 \notin \mathcal{L} & \hat{\delta}(q_0, 01) = q_1 \notin \mathcal{F} \\ & x = 10 \notin \mathcal{L} & \hat{\delta}(q_0, 10) = q_1 \notin \mathcal{F} \\ & x = 11 \in \mathcal{L} & \hat{\delta}(q_0, 11) = q_2 \in \mathcal{F} \end{array}\right\} \text{Base}$$

Passo induttivo. Sia x una stringa con $|x| = n + 1$, dove $n \geq 2$. Allora $x = va$, con $|v| = n$ e $a \in \Sigma$. Per ipotesi induttiva, lo stato raggiunto leggendo v dipende dal numero di 1 che contiene, come nella base:

$$\hat{\delta}(q_0, v) = \begin{cases} q_0 & \text{se } v \text{ non contiene } 1, \\ q_1 & \text{se } v \text{ contiene esattamente un } 1, \\ q_2 & \text{se } v \text{ contiene almeno due } 1. \end{cases}$$

Poiché $\hat{\delta}(q_0, va) = \delta(\hat{\delta}(q_0, v), a)$, basta considerare lo stato raggiunto con v e l'ultimo simbolo a.

1. v non contiene 1, quindi $\hat{\delta}(q_0, v) = q_0$. Sia che alla fine si aggiunga uno 0, sia che si aggiunga un 1, l'automa non va nello stato finale:

   $$\hat{\delta}(q_0, v0) = \delta(q_0, 0) = q_0 \notin \mathcal{F}, \qquad \hat{\delta}(q_0, v1) = \delta(q_0, 1) = q_1 \notin \mathcal{F}.$$

   Ed è corretto, perché $v0$ non contiene 1 e $v1$ ne contiene uno solo: in entrambi i casi $x \notin \mathcal{L}$.
2. v contiene esattamente un 1, quindi $\hat{\delta}(q_0, v) = q_1$. Se alla fine si aggiunge uno 0, l'automa rimane in q_1 e non accetta la stringa: è corretto, perché $v0$ contiene ancora un solo 1 e quindi $x \notin \mathcal{L}$. Se invece l'ultimo simbolo è 1, la stringa contiene due 1 e appartiene a $\mathcal{L}$, e l'automa passa da q_1 a q_2, riconoscendola:

   $$\hat{\delta}(q_0, v0) = \delta(q_1, 0) = q_1 \notin \mathcal{F}, \qquad \hat{\delta}(q_0, v1) = \delta(q_1, 1) = q_2 \in \mathcal{F}.$$

3. v contiene almeno due 1, quindi $v \in \mathcal{L}$ e $\hat{\delta}(q_0, v) = q_2$. Qualunque simbolo si aggiunga, la stringa resta in $\mathcal{L}$ e l'automa rimane in q_2:

   $$\hat{\delta}(q_0, v0) = \delta(q_2, 0) = q_2 \in \mathcal{F}, \qquad \hat{\delta}(q_0, v1) = \delta(q_2, 1) = q_2 \in \mathcal{F}.$$

In tutti i casi lo stato raggiunto da x corrisponde al numero di 1 che contiene, quindi l'ipotesi vale anche per $|x| = n + 1$. In particolare $\hat{\delta}(q_0, x) \in \mathcal{F}$ se e solo se x contiene almeno due 1, ovvero $\mathcal{L}(\mathcal{M}) = \mathcal{L}$. ◇

[1] In realtà, non è mai possibile considerare $|x| = 0$ come unico caso base, perché l'unica stringa di tale lunghezza, ε, o appartiene a un linguaggio o non vi appartiene. Servono, invece, lunghezze tali per cui esistono stringhe sia appartenenti che non.